TABLE OF CONTENTS

CHAPTER NO		TOPIC	PAGE NO
		ABSTRACT	
		LIST OF FIGURES	
		LIST OF ABBREVIATIONS	
1		**INTRODUCTION**	1
	1.1	Objectives	3
	1.2	Problem Statement	5
2		**LITERATURE SURVEY**	7
3		**SYSTEM DESIGN**	11
	3.1	Existing System	11
	3.1.1	Disadvantages of Existing Systems	12
	3.2	Proposed System	13
	3.2.1	Architecture Diagram	15
	3.2.2	Explanation	16
	3.2.3	Advantages of the Proposed System	17
	3.2.4	Workflow Diagram	18
4		**SYSTEM REQUIREMENT**	19
	4.1	Hardware Requirement	19
	4.1.1	Hardware Description	19
	4.2	Software Requirement	20
	4.2.1	Software Description	20
5		**MODULE DESCRIPTION**	22
	5.1	Module List	22
	5.1.1	Module Description	22
	5.2	Unified Architecture of TrackBot Modules	30

6		**RESULT AND DISCUSSION**	32
	6.1	Result	32
	6.2	Discussion	33
7		**CONCLUSION AND FUTURE ENHANCEMENT**	34
	7.1	Conclusion	34
	7.2	Future Enhancement	35
		APPENDIX A: CODING	37
		APPENDIX B: SCREENSHOT	59
		REFERENCES	63

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
3.2.1	Proposed system workflow	15
3.2.2	Working Flowchart	18
5.1.1	Track Image Acquisition Module Architecture	23
5.1.2	Rail Localization & Gauge Measurement Module Architecture	24
5.1.3	Crack Detection Module Architecture	25
5.1.4	Obstacle Detection Module Architecture	26
5.1.5	Motion Control & Navigation Module Architecture	27
5.1.6	Localization & Defect Logging Module Architecture	27
5.1.7	Alert, Dashboard & Report Module Architecture	28
5.1.8	Safety & Fail-safe Module Architecture	29
5.1.9	TrackBot System Module Integration Diagram	30

LIST OF ABBREVIATIONS

S.NO	ABBREVIATION	MEANING
1	**CV**	Computer Vision
2	**GPS**	Global Positioning System
3	**IMU**	Inertial Measurement Unit
4	**PWM**	Pulse Width Modulation
5	**GSM**	Global System for Mobile Communication
6	**UART**	Universal Asynchronous Receiver Transmitter
7	**BG**	Broad Gauge
8	**HSV**	Hue Saturation Value
9	**RMS**	Root Mean Square
10	**UI**	User Interface

CHAPTER 1

INTRODUCTION

Railways form the backbone of passenger and freight transport in India, carrying millions of people and enormous volumes of goods every day across one of the largest rail networks in the world. The safety of this network depends directly on the condition of the track. Small defects such as transverse cracks on the rail head, widening or tightening of the track gauge, and obstructions lying on the line can grow into rail fractures and derailments if they are not found in time. Regular, reliable and well-documented inspection of the track is therefore one of the most important maintenance activities of any railway administration.

In India, routine track inspection is still largely manual. Keymen and trackmen patrol their beat on foot or push a small trolley along the rails, checking the track visually and measuring the gauge with hand tools at selected points. A single inspector can cover only a few kilometres in a shift, the quality of the inspection depends on the attention and experience of that person, and the results are usually written on paper without a photograph or an exact location. Tiny hairline cracks are easily missed, especially in poor light, in rain or at the end of a long patrol. Specialised ultrasonic flaw-detection trolleys and track recording cars exist, but they are expensive, few in number and run only periodically on each section.

TrackBot is proposed as a practical answer to this problem. TrackBot is a low-cost, autonomous, rail-mounted inspection robot that runs on the track like a miniature train or go-kart. It is driven by DC geared motors on flanged wheels and travels along the rails by itself at a controlled inspection speed. A down-looking camera with its own LED lighting continuously views both rails, while a forward-looking camera and ultrasonic sensors watch the track ahead. An on-board Raspberry Pi processes every frame with computer vision to locate both rails, measure the track gauge and detect cracks on the rail head, and an ESP32 controller handles the motors, sensors and safety functions.

Whenever a defect is found, TrackBot records its type, severity, rail side, chainage in kilometres and GPS position together with a close-up photograph. Critical defects raise an

immediate alert to the control room and the defect spot can be marked with paint. When an obstacle such as a boulder, a fallen log or a track worker is seen on the line, the robot slows down early and performs an emergency stop before reaching it, and resumes inspection automatically once the track is clear. All of this information is shown live on a web-based control-room dashboard that can be opened on any laptop or mobile phone over Wi-Fi, and an inspection report is generated at the end of every run.

TrackBot does not attempt to replace the railway engineer; instead, it augments the inspection team by doing the repetitive, tiring part of the work consistently and recording every metre of track with photographic evidence. It operates as an assistive layer between the maintenance staff and the track, bringing defects to their attention with exact locations so that limited manpower can be directed where it is needed most. Unlike traditional manual patrolling, which depends on human discipline and endurance, TrackBot integrates inspection into a repeatable, automated process that can be run during every maintenance block.

Technologically, the system integrates several core components: a track image acquisition unit, a rail localization and gauge measurement algorithm, a crack detection pipeline based on morphological image processing, an obstacle detection unit that fuses camera and ultrasonic data, a motion control system with closed-loop speed control, a localization and defect logging unit, and a live dashboard with alerts and reports. A separate safety layer running on the ESP32 makes sure that the robot stops even if the main computer fails. All these components work together in real time, with every frame processed while the robot is moving.

The introduction of TrackBot also opens possibilities for future enhancements. Deep learning models such as YOLO can be added for missing fasteners and broken sleepers, ultrasonic probes can detect internal flaws, and the defect records can be connected to railway maintenance management software. TrackBot thus contributes to safer railways by making track inspection faster, more objective, better documented and far less expensive than existing mechanised methods.

1.1 Objectives

The primary objective of **TrackBot – Autonomous Railway Track Inspection System** is to automate routine track inspection so that defects are found earlier, more reliably and with exact location and photographic evidence. The project aims to reduce the dependence on manual patrolling, where an inspector walks or pushes a trolley over the track and judges its condition by eye. By deploying a self-driving rail-mounted robot that inspects every frame of the track, TrackBot seeks to make the inspection process consistent, repeatable and independent of human fatigue.

Another objective is to achieve **accurate detection of rail-head cracks**. The system must distinguish genuine transverse cracks from harmless marks such as rust patches, grease stains and shadows. It must also grade each crack by how far it runs across the rail head into MINOR, MAJOR and CRITICAL categories, so that maintenance staff can prioritise their work. False alarms must be kept to a minimum because a system that reports too many non-existent defects quickly loses the trust of its users.

A further objective is the **continuous measurement of track gauge**. On Indian Broad Gauge the nominal distance between the inner faces of the rails is 1676 mm. TrackBot aims to measure the gauge in every frame using computer vision alone, without contact gauges, and to report every stretch where the gauge goes beyond the permitted tolerance as a wide gauge or tight gauge fault, together with its start point, length and worst value.

Safety of the robot and of people working on the track is another key goal. TrackBot aims to **detect obstacles on the line** such as boulders, fallen trees, animals and track workers using a front camera for long-range warning and ultrasonic sensors for accurate short-range distance. The robot must slow down when an obstacle is seen far ahead, stop safely before reaching it, alert the control room, and resume automatically once the path has been clear for a set time.

Precise localization of every defect is an equally important objective. Each finding must be tagged with its chainage in kilometres computed from the wheel encoder, its GPS coordinates,

the rail side and a close-up photograph. Detections of the same defect from consecutive frames must be merged into a single record so that the final defect register is clean and directly usable by the maintenance gang.

The project also aims to provide a **live control-room dashboard**. The operator must be able to see both camera feeds, speed, chainage, gauge, obstacle range, vibration and battery level, a track defect map and an event log in real time from any browser. The operator must also be able to start, stop or manually drive the robot remotely, and download a complete inspection report in CSV and PDF formats at the end of the run.

Fail-safe operation is a further objective. TrackBot must stop by itself if an obstacle is too close, if the emergency stop button is pressed, or if the main computer stops sending commands. These functions are therefore implemented on the ESP32 microcontroller with a watchdog timer, so that a software failure on the Raspberry Pi can never leave the robot running uncontrolled on the track.

Low cost and portability form another essential objective. The complete robot is designed to be built from commonly available components such as a Raspberry Pi, an ESP32, USB cameras, ultrasonic sensors and DC geared motors, at a total cost of roughly twenty to twenty-five thousand rupees. It must be light enough for two people to place on and remove from the track during a maintenance block, and simple enough to be operated by field staff after a short training.

Research contribution is also an objective of this work. TrackBot demonstrates how classical computer vision techniques, sensor fusion and embedded control can be combined into a complete inspection system. The project includes a track simulator with known ground truth, which allows the detection accuracy, false alarm rate and gauge measurement error to be measured objectively, and provides a platform on which future deep learning models can be evaluated.

Ultimately, the overarching goal of TrackBot is to contribute to safer railway operation by finding track defects before they cause accidents. By combining autonomous movement, intelligent image analysis, reliable safety functions and clear reporting, the project offers a comprehensive and affordable solution to a long-standing maintenance challenge. This objective framework ensures continuous improvement of track condition monitoring, better use of maintenance manpower and reliable performance across different track sections.

1.2 Problem Statement

The safe running of trains depends on the condition of the rails, the sleepers, the fastenings and the track geometry. Rail-head cracks grow under repeated wheel loads and temperature changes and can eventually lead to a complete rail fracture. Gauge widening caused by worn or missing fastenings and damaged sleepers allows wheels to drop between the rails. Obstructions such as boulders, fallen trees and animals on the line are another frequent cause of accidents. All of these hazards must be detected early, located precisely and repaired before a train reaches them.

At present, most routine inspection on Indian Railways is carried out manually. A keyman patrols his beat on foot every day and a push-trolley inspection is carried out periodically by supervisors. The inspector checks the rails by eye and measures the gauge with a hand gauge at selected points. This process is slow, covers only a few kilometres per shift, and depends heavily on the concentration of the individual. Hairline cracks on a shining rail head are difficult to see, particularly in poor light, rain, fog or at night, and the gauge between measurement points is never checked at all.

The records produced by manual inspection are also weak. Observations are generally written on paper with an approximate location and without photographic evidence. It is difficult for supervisors to verify what was inspected, to compare the condition of the same spot over time, or to plan maintenance work based on reliable data. Important defects may be reported late or not at all, and there is no immediate alert to the control room when a critical defect is found.

Mechanised alternatives exist but do not fill this gap. Ultrasonic flaw-detection trolleys are operated manually and still need trained staff to walk with them. Track recording cars and integrated measurement trains are very expensive, are available in small numbers and therefore visit each section only a few times a year. Low-cost student prototypes reported in the literature usually rely on a single IR or ultrasonic threshold sensor; they can indicate that something is wrong but cannot measure the size of a crack, cannot measure the gauge and provide no image of the defect.

Safety of the inspection activity itself is another concern. Staff working on the track are exposed to moving trains, and an automated inspection device must not become a new hazard. Any robot placed on the track must be able to recognise obstacles and people in its path, stop safely before reaching them, and stop by itself if its control software fails.

Therefore, the core problem is the absence of an affordable, autonomous and intelligent track inspection system that can continuously detect rail cracks, measure track gauge and handle obstacles in real time, while recording every defect with an exact location and photographic evidence and reporting it immediately to the control room. TrackBot is designed to address this gap.

CHAPTER 2

LITERATURE SURVEY

Year: 2021

Authors: T. Pamuła and W. Pamuła

Title: Detection of Safe Passage for Trains at Rail Level Crossings Using Deep Learning

Summary: This research addresses the safety of trains at rail level crossings by analysing video from the cameras installed at the crossing. The authors use a deep learning method to decide whether the area required for the safe passage of a train is free of vehicles, people and other objects. The method was validated on recordings from several level-crossing sites in Poland under day and night conditions and different weather. The reported recall reaches 0.98 while using significantly reduced processing resources, which makes the approach suitable for embedded hardware. The authors suggest that the output can be combined with other sensor data in train control systems. The work shows that camera-based detection of obstructions in the track area is practical, which supports the front-camera obstacle detection used in TrackBot.

Year: 2021

Authors: D. Ristić-Durrant, M. Franke and K. Michels

Title: A Review of Vision-Based On-Board Obstacle Detection and Distance Estimation in Railways

Summary: This paper presents a comprehensive review of on-board obstacle detection methods for railway applications. The authors survey the sensors used for this purpose and focus on vision sensors, dividing the methods into traditional computer vision approaches and artificial intelligence based approaches. They point out that obstacle detection research is well developed for road vehicles but has been far less extensive for railways, and they highlight the importance of estimating the distance to a detected obstacle rather than only detecting it. The review identifies track region detection, object detection and distance estimation as the key building blocks of such a system. These findings directly guided the TrackBot design, which

first finds the track corridor, then detects objects inside it and estimates their distance from the camera geometry and the ultrasonic sensor.

Year: 2021

Authors: D. Zhang, K. Song, Q. Wang, Y. He, X. Wen and Y. Yan

Title: Two Deep Learning Networks for Rail Surface Defect Inspection of Limited Samples With Line-Level Label

Summary: The authors address a practical problem in deep learning based rail inspection: most methods require thousands of labelled defect samples, and collecting and annotating such samples is laborious and costly. They propose two deep learning networks that can be trained with limited samples using simpler line-level labels instead of detailed pixel-level annotation. Experiments on rail surface images show that the networks can locate surface defects effectively despite the small training set. The work highlights that annotated rail defect data is scarce, which is one reason why TrackBot uses a training-free morphological crack detector in its first version and keeps deep learning as a future enhancement once enough field images have been collected.

Year: 2021

Authors: D. Zheng, L. Li, S. Zheng, X. Chai, S. Zhao, Q. Tong, J. Wang and L. Guo

Title: A Defect Detection Method for Rail Surface and Fasteners Based on Deep Convolutional Neural Network

Summary: This study proposes a three-stage pipeline for inspecting rail surfaces and fasteners from track images. First, the rails and fasteners in the image are localised using an improved YOLOv5 framework. Second, a Mask R-CNN based model detects surface defects on the rail and segments the defect area. Third, a ResNet based classifier determines the state of each fastener. The method was tested on images of ballasted and ballastless track from the Shijiazhuang–Taiyuan high-speed railway line. The work demonstrates the value of first localising the rail and then searching for defects only inside it, an approach that TrackBot also follows by building a rail mask before running crack detection.

Year: 2017

Authors: J. Gan, Q. Li, J. Wang and H. Yu

Title: A Hierarchical Extractor-Based Visual Rail Surface Inspection System

Summary: The authors present a visual rail surface inspection system that captures images of the rail surface and processes them through a hierarchical framework consisting of a coarse extractor and a fine extractor. The coarse extractor works by studying the rail background rather than the defects themselves, focusing on finding the background modes, so that anything that does not match the normal rail surface is treated as a candidate defect. The fine extractor then combines longitudinal context with transversal prior information to reduce interference from other irregularities. The paper shows that using the regular appearance of the rail as prior knowledge reduces false alarms, a principle reflected in TrackBot's shape test that rejects round stains and keeps only line-like cracks running across the rail head.

Year: 2017

Authors: X. Gibert, V. M. Patel and R. Chellappa

Title: Deep Multitask Learning for Railway Track Inspection

Summary: This paper applies deep convolutional neural networks to the inspection of railway track components using images captured from a moving inspection vehicle. Instead of training a separate network for every task, the authors use a multitask learning approach in which a single network learns several related inspection tasks, such as recognising fasteners and detecting defects in sleepers. Sharing features between tasks improves the use of limited training data and reduces computation. The work demonstrates that automated visual inspection from a moving platform is feasible and that a single camera stream can serve several inspection functions, which supports the TrackBot design in which one down-looking camera is used for both gauge measurement and crack detection.

Year: 2017

Authors: Y. Santur, M. Karakose and E. Akin

Title: A New Rail Inspection Method Based on Deep Learning Using Laser Cameras

Summary: The authors propose a hardware and software architecture for railway surface inspection based on 3D laser cameras combined with deep learning. The laser cameras capture the surface profile of the rail, and the deep learning model identifies surface defects such as fractures, scouring and wear. The approach aims to overcome the limitations of manual inspection and to allow faster and more objective inspection. While laser cameras provide accurate depth information, they increase the cost and complexity of the system. TrackBot instead uses an ordinary camera with controlled LED lighting to keep the cost low, and lists a laser line profiler as a future enhancement for measuring rail wear.

Year: 2016

Authors: S. Faghih-Roohi, S. Hajizadeh, A. Núñez, R. Babuska and B. De Schutter

Title: Deep Convolutional Neural Networks for Detection of Rail Surface Defects

Summary: This work investigates deep convolutional neural networks for detecting rail surface defects from images captured by a camera on a measurement train. The rail images are cropped around the rail head and classified into normal rail and different classes of surface defects. The authors compare network structures of different sizes and analyse the trade-off between classification accuracy and training time. The study is one of the early demonstrations that rail-head images taken from a moving vehicle contain enough information to detect surface defects automatically. It motivated the TrackBot approach of cropping the analysis to the rail head and grading defects by their size, so that maintenance can be prioritised.

CHAPTER 3

SYSTEM DESIGN

3.1 Existing System

Existing systems for railway track inspection in India remain limited in coverage, frequency and documentation. The most common method is manual patrolling, in which a keyman walks his allotted section every day and inspects the rails, fastenings and sleepers visually. Supervisors carry out push-trolley inspections periodically, moving along the line on a small manually pushed trolley and checking the track by eye and with hand tools. These methods rely entirely on human attention, are slow, and record observations on paper without photographs or exact positions.

Track gauge in the existing system is measured with a mechanical track gauge placed by hand across the rails at selected locations. The measurement is accurate at that spot, but the stretches between measurement points are never checked, and gradual gauge widening caused by worn fastenings or damaged sleepers can remain unnoticed until it becomes dangerous. Recording the readings, computing deviations and reporting them is also done manually, which takes time and is prone to errors.

Ultrasonic flaw-detection trolleys are used to find internal defects in rails. These trolleys are effective for internal flaws but are pushed manually at walking speed by trained operators, so their coverage per day is limited. Track recording cars and integrated measurement trains provide detailed geometry measurements at high speed, but they are expensive, few in number and must be scheduled for each section, so a given stretch of track is recorded only a few times in a year. Between these visits, the condition of the track is known only from manual patrolling.

Several low-cost automatic crack detection prototypes have been reported by students and researchers. Most of them use an Arduino or NodeMCU with an IR sensor or an ultrasonic sensor mounted over each rail. When the reflected signal changes beyond a fixed threshold, the

device assumes that a crack is present, stops, and sends an SMS with the GPS location through a GSM module. These systems are inexpensive, but they can detect only large gaps, cannot measure the size or severity of a crack, cannot measure the gauge, and are easily triggered by dirt, grease or uneven rail surfaces.

Obstacle detection on the track is generally not part of existing inspection devices. Some prototypes include a single ultrasonic or PIR sensor that stops the device when something is very close, but there is no long-range detection, no distance estimation and no classification of the situation. At railway level crossings, CCTV and radar systems are being studied for detecting obstructions, but these are fixed installations and do not help an inspection device moving along the line.

Data management is another weak point of the existing system. Inspection results are usually maintained in registers, and there is no live view of what is being inspected, no central defect map and no automatic alert when a critical defect is found. Supervisors cannot easily verify the work done or compare the condition of the same location over time. Consequently, existing systems fail to provide the continuous, objective and well-documented inspection that modern railway maintenance requires.

3.1.1 Disadvantages of Existing Systems

1. **Dependence on Human Attention** – Manual patrolling relies on the concentration and experience of the inspector, which drops with fatigue.
2. **Slow Coverage** – A keyman or push trolley covers only a few kilometres per shift.
3. **Missed Hairline Cracks** – Small cracks on a bright rail head are hard to see, especially in poor light.
4. **No Continuous Gauge Check** – Gauge is measured only at selected points with a hand gauge.
5. **Paper-based Records** – Observations are written manually without photographs.

6. **Approximate Location** – Defect positions are noted roughly, making repair teams search for them.
7. **No Real-time Alert** – Critical defects are reported late; the control room is not informed at once.
8. **High Cost of Mechanised Systems** – Track recording cars and measurement trains are very expensive.
9. **Infrequent Mechanised Inspection** – Recording cars visit each section only a few times a year.
10. **Threshold-only Prototypes** – IR / ultrasonic prototypes detect only large gaps and give no crack size.
11. **High False Alarms** – Simple sensors are triggered by dirt, grease and rust on the rail.
12. **No Obstacle Awareness** – Inspection devices cannot see obstacles ahead or estimate their distance.
13. **No Fail-safe Behaviour** – Prototypes do not stop safely if their controller fails.
14. **No Central Defect Map** – There is no live dashboard or consolidated view of defects along the section.
15. **Difficult Verification** – Supervisors cannot easily verify what was inspected and when.

3.2 Proposed System

The proposed system, **TrackBot**, introduces an autonomous rail-mounted inspection robot that combines computer vision, sensor fusion and embedded control to inspect the track continuously and report defects in real time. Unlike manual patrolling, which depends on the attention of an inspector, TrackBot examines every frame of the track with the same algorithm and records every finding with an exact position and a photograph. Unlike threshold-based prototypes, it measures the size of each crack, measures the track gauge and watches the track ahead for obstacles.

TrackBot runs on the rails on flanged wheels driven by two 12 V DC geared motors. A down-looking camera mounted under the chassis with an LED ring light captures a top view of both rails. In every frame the system locates both rail heads using brightness profiles, measures the gauge between their inner faces and searches the rail head for cracks using a morphological black-hat filter followed by a shape test. Each crack is graded as MINOR, MAJOR or CRITICAL depending on how far it runs across the rail head, while round rust and grease stains are rejected.

A forward-looking camera and ultrasonic sensors form the obstacle detection unit. The camera detects objects inside the track corridor up to about twenty-five metres ahead and estimates their distance from the camera geometry, allowing the robot to slow down early. The ultrasonic sensors measure the distance accurately at short range, and the robot performs an emergency stop at two metres. After the obstacle is removed and the track has remained clear for two seconds, inspection resumes automatically. This sensor fusion approach combines the long range of the camera with the accuracy of the ultrasonic sensor.

A wheel encoder gives the distance travelled, which is converted into chainage in kilometres, and a GPS module gives the coordinates of each defect. Detections of the same crack in consecutive frames are merged into a single defect record with its severity, rail side, chainage, GPS position and a close-up photograph. Critical cracks and obstructions trigger an immediate alert to the control room and can fire a paint marker so that the maintenance gang can find the spot easily.

The control-room dashboard is a web application served by the robot itself. It shows both camera feeds with detection overlays, speed, chainage, gauge, obstacle range, vibration, battery level, a track defect map with the gauge profile, an event log and a defect register with photographs. The operator can start and stop the robot, change the cruise speed and drive it manually from any browser on the same network. At the end of the run, an inspection report is generated in CSV, JSON and printable HTML/PDF formats.

Safety is built into two layers. The Raspberry Pi runs the main control state machine, while the ESP32 microcontroller independently enforces an ultrasonic hard-stop, monitors the emergency stop button and runs a one-second watchdog. If commands from the Raspberry Pi stop arriving, the motors are stopped automatically. This ensures that a software failure on the main computer can never leave the robot moving uncontrolled on the track.

The system architecture consists of several interconnected modules that collaboratively process camera and sensor data. The image acquisition module captures synchronised frames; the rail localization and crack detection modules analyse the track image; the obstacle detection module analyses the scene ahead; the motion control module decides the robot speed; the defect logging module creates geo-tagged records; and the dashboard module presents everything to the operator. By separating these functions, TrackBot remains easy to test, extend and maintain.

3.2.1 Architecture Diagram

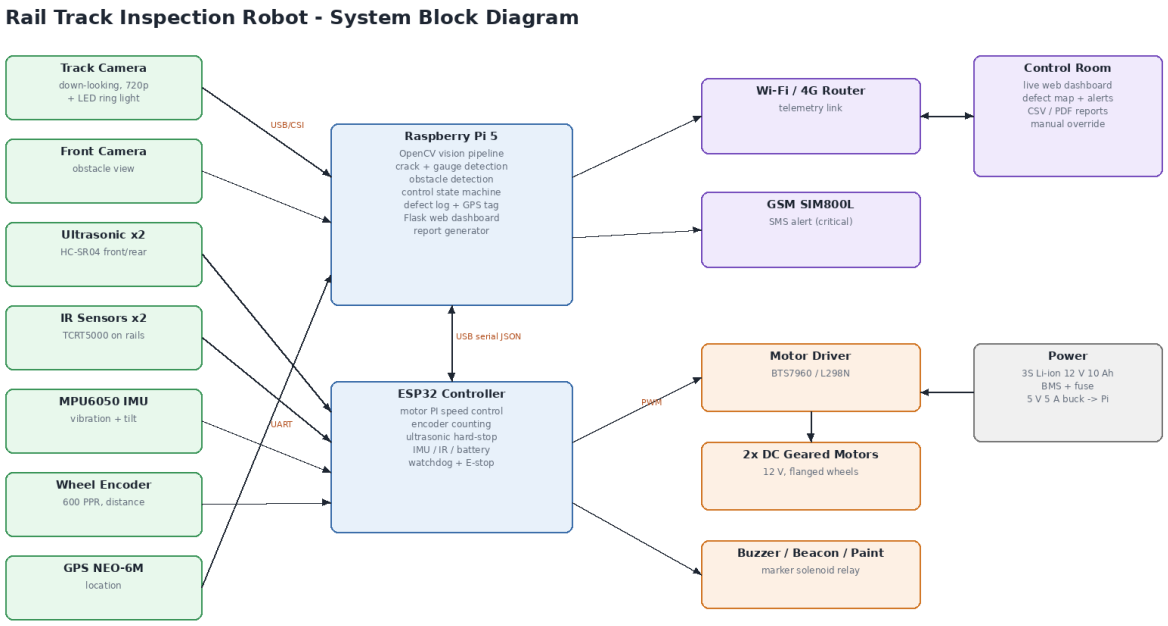


Fig 3.2.1 Proposed system workflow

3.2.2 Explanation

The architecture diagram of TrackBot illustrates the complete flow of information from the sensors to the control room. On the left are the sensing elements: the down-looking track camera with LED lighting, the front camera, front and rear ultrasonic sensors, IR sensors over each rail, the MPU6050 inertial measurement unit, the wheel encoder and the GPS module. The cameras and GPS are connected to the Raspberry Pi 5, which acts as the main processing unit, while the encoder, ultrasonic sensors, IR sensors and IMU are connected to the ESP32 controller.

The Raspberry Pi runs the OpenCV vision pipeline for rail localization, gauge measurement, crack detection and obstacle detection, together with the control state machine, the defect log and the Flask web dashboard. It communicates with the ESP32 over USB serial using short JSON messages: the ESP32 sends encoder distance, speed, ultrasonic range, battery voltage, vibration and tilt twenty times per second, and the Raspberry Pi sends drive, stop and marker commands.

The ESP32 drives the two DC geared motors through a BTS7960 motor driver using PWM signals and a proportional-integral speed controller with encoder feedback. It also controls the buzzer, the warning beacon and the relay of the paint-marker solenoid. Power is supplied by a 3S lithium-ion battery pack with a battery management system and fuse, and a 5 V buck converter supplies the Raspberry Pi.

On the communication side, the Raspberry Pi connects to a Wi-Fi or 4G router so that the control room can open the live dashboard, view the defect map and download reports, and an optional GSM module can send SMS alerts for critical defects. In this way, the architecture links sensing, processing, actuation and reporting into a single coordinated system in which every detected defect reaches the maintenance team with its location and evidence.

3.2.3 Advantages of the Proposed System

1. **Continuous Inspection** – Every frame of the track is analysed while the robot is moving.
2. **Accurate Crack Detection** – Cracks are detected and graded by severity with very few false alarms.
3. **Continuous Gauge Measurement** – Gauge is measured in every frame, not only at selected points.
4. **Obstacle Detection and Auto-stop** – The robot slows down early and stops safely before obstacles.
5. **Exact Location** – Every defect is tagged with chainage, GPS coordinates and rail side.
6. **Photographic Evidence** – A close-up photo of each defect is stored in the report.
7. **Real-time Alerts** – Critical defects are reported to the control room immediately.
8. **Low Cost** – Built from common components for about twenty-five thousand rupees.
9. **Fail-safe Design** – Watchdog, hard-stop and E-stop on the ESP32 keep the robot safe.
10. **Remote Monitoring** – Live dashboard and reports are available on any phone or laptop.

3.2.4 Workflow Diagram

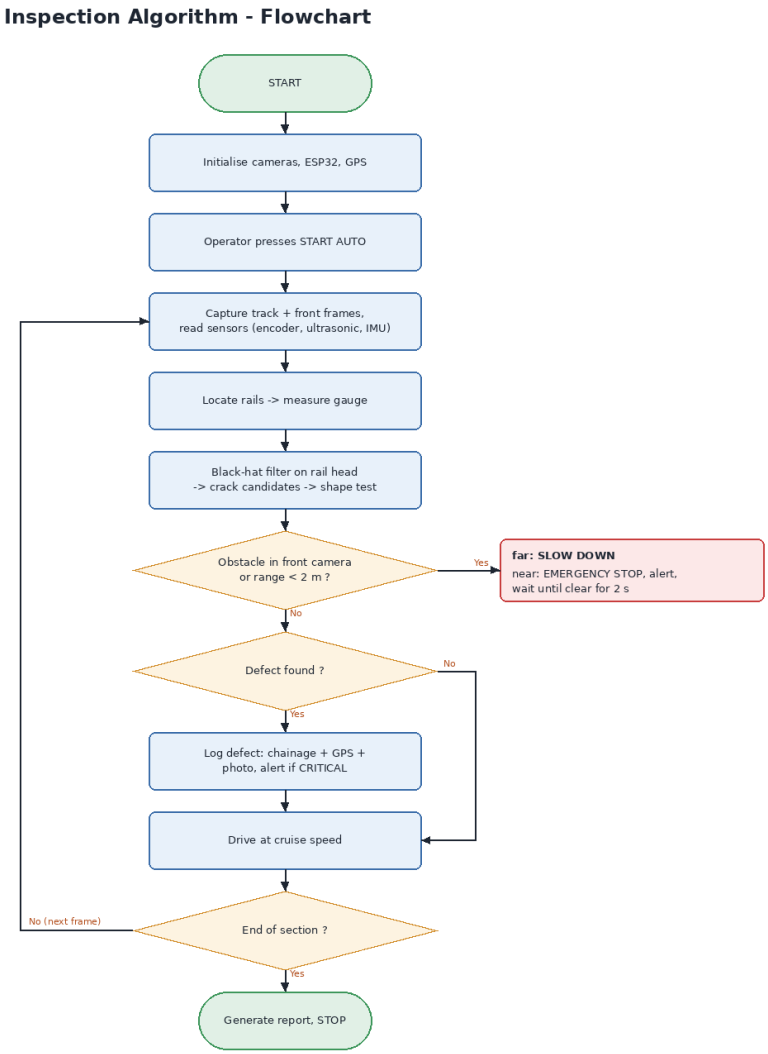


Fig 3.2.2 Working Flowchart

CHAPTER 4

SYSTEM REQUIREMENT

4.1 Hardware Requirement

PROCESSOR : RASPBERRY PI 5 (4 GB RAM), ESP32 DEVKIT

CAMERA : 2 × USB / PI CAMERA (720p) WITH LED RING LIGHT

SENSORS : HC-SR04 ULTRASONIC × 2, TCRT5000 IR × 2, MPU6050, ROTARY ENCODER, NEO-6M GPS

DRIVE : 2 × 12 V DC GEARED MOTORS, BTS7960 MOTOR DRIVER

POWER : 3S LI-ION 12 V 10 AH BATTERY WITH BMS

4.1.1 Hardware Description

1. Processor : Raspberry Pi 5 and ESP32

The **Raspberry Pi 5** is a compact single-board computer with a quad-core 64-bit ARM processor. It runs the computer vision pipeline, the control state machine, the defect log and the web dashboard. The **ESP32** is a dual-core microcontroller with Wi-Fi that handles real-time tasks: motor speed control, encoder counting, ultrasonic measurement and the safety watchdog.

2. Cameras and Sensors

A down-looking camera with an LED ring light captures the rails for crack detection and gauge measurement, and a front camera watches the track ahead. Ultrasonic sensors measure obstacle distance, IR sensors check rail presence, the MPU6050 measures vibration and tilt, the encoder measures distance, and the GPS module provides coordinates.

3. Drive and Power

Two 12 V DC geared motors on flanged wheels move the robot along the rails through a BTS7960 driver. A 3S lithium-ion battery pack with a BMS powers the system, with a 5 V

buck converter for the Raspberry Pi. An E-stop button, buzzer, beacon and paint-marker relay complete the hardware.

4.2 Software Requirement

PYTHON, OPENCV, FLASK, ARDUINO IDE (ESP32)

4.2.1 Software Description

1. Python Technology

Python is an interpreted, high-level, general-purpose programming language. It supports multiple programming paradigms, including procedural, object-oriented and functional programming. **Python** is often described as a "batteries included" language due to its comprehensive standard library, and in TrackBot it is used for the vision pipeline, control logic and dashboard server.

2. OpenCV, Flask and Arduino

OpenCV provides the image processing functions used for morphology, thresholding, connected components and colour conversion. Flask serves the web dashboard, live camera streams and REST API. The ESP32 firmware is written in C++ using the Arduino IDE with the ESP32 core.

The software stack offers a wide range of functionality, including:

- Easy to Learn and Use
- Real-time Image Processing (OpenCV)
- Interpreted Language
- Cross-platform
- Free and Open Source
- Lightweight Web Framework (Flask)
- Embedded C++ Firmware (Arduino)
- Large Standard Library

CHAPTER 5

MODULE DESCRIPTION

5.1 Module List

1. Track Image Acquisition Module
2. Rail Localization & Gauge Measurement Module
3. Crack Detection Module
4. Obstacle Detection Module
5. Motion Control & Navigation Module
6. Localization & Defect Logging Module
7. Alert, Dashboard & Report Module
8. Safety & Fail-safe Module

5.1.1 Module Description

1. Track Image Acquisition Module

The Track Image Acquisition Module is responsible for continuously capturing images of the track while TrackBot is moving. Its primary role is to provide clear, evenly lit images of both rails and of the scene ahead so that the analysis modules can work reliably. This module acts as the system's "eyes", supplying every other module with visual information about the track.

The module uses two cameras. The down-looking track camera is mounted under the chassis at a fixed height and views both rails from above, covering about two metres of track in each frame. An LED ring light around the camera provides constant illumination, so that the appearance of the rail head does not change with sunlight, shadows or time of day. The forward-looking camera is mounted at the front of the robot and captures the track ahead for obstacle detection.

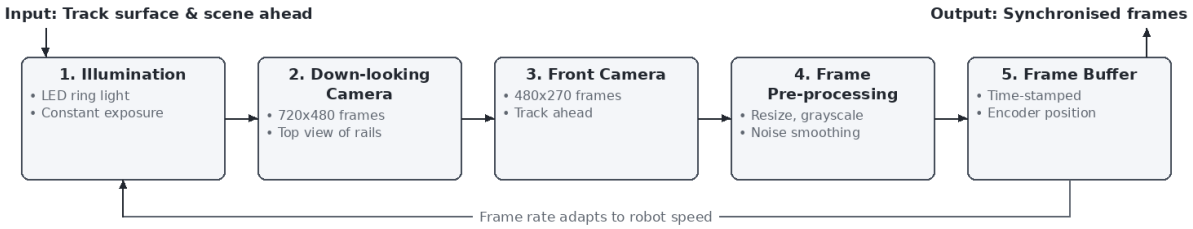


Fig 5.1.1 Track Image Acquisition Module Architecture

Each frame is resized to the working resolution, converted to grayscale where needed and smoothed lightly to reduce sensor noise. The frame is time-stamped and paired with the current encoder position, so that every pixel row of the image can later be converted into a position along the track. The output of this module is a pair of synchronised frames, the track image and the front image, which serve as the foundational input for all subsequent processing steps in TrackBot.

2. Rail Localization & Gauge Measurement Module

The Rail Localization and Gauge Measurement Module finds both rails in every track image and measures the distance between them. Its main goal is to know exactly where the rail heads are, so that crack detection can be restricted to the rail surface, and to measure the track gauge continuously along the whole section instead of only at selected points.

To achieve this, the image is divided into eight horizontal bands. In each band the average brightness of every column is computed, giving a brightness profile across the track. The polished rail heads are the brightest features in the image, so after subtracting a smoothed version of the profile they appear as bright runs about 72 mm wide. The run closest to the expected position of each rail is selected, and bands whose rail positions disagree with the median of the frame, for example because of debris or glare, are discarded.

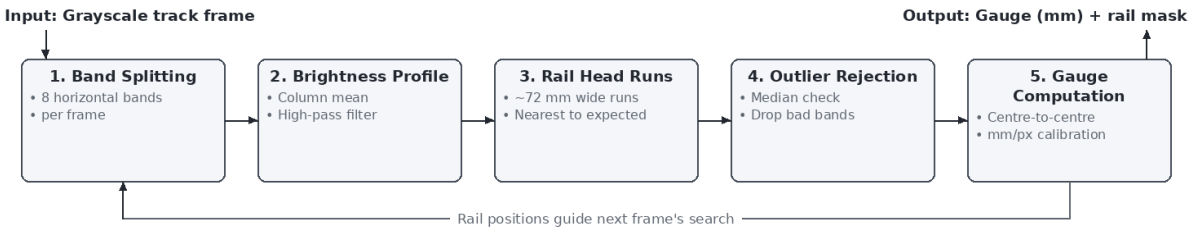


Fig 5.1.2 Rail Localization & Gauge Measurement Module Architecture

The gauge is then calculated from the centres of the two rail heads: the centre-to-centre distance in pixels is multiplied by the calibrated millimetres-per-pixel factor and the width of one rail head is subtracted to obtain the distance between the inner faces. The median over all bands is used as the gauge of the frame. The output of this module is the measured gauge in millimetres and a rail mask that marks the rail-head region for the next module. Any stretch where the gauge differs from the nominal 1676 mm by more than the tolerance is reported as a wide or tight gauge fault.

3. Crack Detection Module

The Crack Detection Module searches the rail head for transverse cracks. A crack appears in the image as a thin dark line running across the bright rail head. The input to this module is the grayscale track frame together with the rail mask produced by the previous module, so that only the rail surface is analysed and the ballast, sleepers and fastenings are ignored.

The module first applies a small Gaussian blur to reduce noise. It then applies a morphological black-hat transform, which is the difference between a morphological closing of the image and the image itself, using a narrow vertical structuring element. The closing fills in thin dark gaps, so the black-hat result is bright exactly where thin dark lines cross the rail. This result is thresholded, masked to the rail head and grouped into connected components, each of which is a crack candidate.



Fig 5.1.3 Crack Detection Module Architecture

Each candidate is then checked with a shape test. A genuine crack runs across the rail, so its length across the head must be at least thirty-five percent of the head width and its width must be at least 1.8 times its height. Rust patches and grease stains are roughly round and therefore fail this test, which keeps false alarms low. The severity of a crack is graded from its length across the rail head: MINOR from 35 percent, MAJOR from 55 percent and CRITICAL from 80 percent. The output of this module is a list of crack detections with their rail side, position in the image, length and severity.

4. Obstacle Detection Module

The Obstacle Detection Module watches the track ahead and determines whether anything is blocking the robot's path, and how far away it is. Obstacles on the line may include boulders, fallen trees or logs, animals and track workers. Detecting them early allows the robot to slow down smoothly instead of braking hard, and detecting them reliably at short range ensures that it never collides with them.

The input to this module is the front camera image and the distance reported by the ultrasonic sensor. Using the known camera height and focal length, a polygon covering the track corridor is projected onto the image. Inside this corridor, pixels whose colour differs from the grey ballast, sleepers and rails are segmented using the HSV colour space, while sky and vegetation colours are removed. Morphological operations clean the mask and connected component analysis groups the pixels into objects.



Fig 5.1.4 Obstacle Detection Module Architecture

For each object whose base lies on the track bed, the distance is estimated from the image row where it touches the ground using the flat-ground camera model d = f·h / (y − y_horizon). The camera estimate is used for early warning, and the ultrasonic sensor, which is accurate up to about four metres, confirms the distance at short range. The output of this module is the obstacle distance and a flag that tells the control module whether to slow down or stop.

5. Motion Control & Navigation Module

The Motion Control and Navigation Module decides how fast TrackBot should move and makes sure the motors achieve that speed. Its purpose is to keep the robot moving at a steady inspection speed for good image quality, to react correctly to obstacles and operator commands, and to stop at the end of the section.

The decision part is a state machine running on the Raspberry Pi with the states Idle, Inspecting, Slowing, Obstacle Stop, Paused, Manual and Completed. In the Inspecting state the target speed is the cruise speed of 1.2 m/s. When the camera sees an obstacle within fifteen metres the state changes to Slowing and the speed is reduced to forty-five percent. When the obstacle is within two metres the state changes to Obstacle Stop and the target speed becomes zero. Once the track has remained clear for two seconds the robot returns to Inspecting automatically.

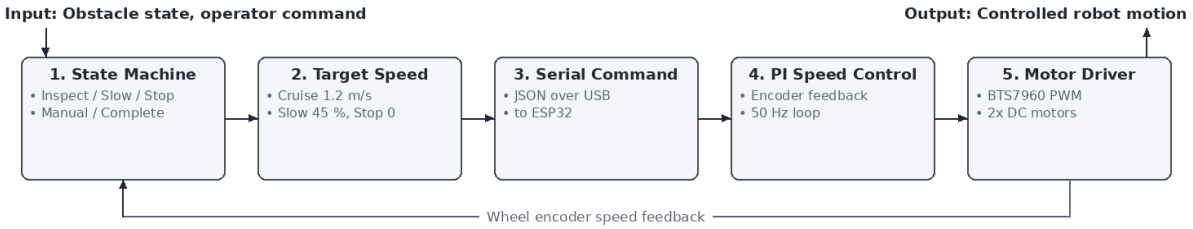


Fig 5.1.5 Motion Control & Navigation Module Architecture

The target speed is sent to the ESP32 as a JSON command over USB serial. On the ESP32, a proportional-integral controller with feed-forward compares the target speed with the speed measured by the wheel encoder fifty times per second and adjusts the PWM signal to the BTS7960 motor driver accordingly. The output of this module is smooth, controlled motion of the robot along the track, with manual override available to the operator at any time.

6. Localization & Defect Logging Module

The Localization and Defect Logging Module converts the detections of individual frames into a clean list of unique, geo-tagged defects. Without this module, the same crack would be reported many times, once for every frame in which it is visible, and its position would not be known precisely.

The position of every detection is computed from the encoder distance of the frame plus the offset of the detection from the centre of the image, converted with the millimetres-per-pixel factor. A crack must be seen in at least two frames before it is accepted, which removes random noise, and detections on the same rail within thirty centimetres are merged into one defect. The severity of the defect is taken from the strongest views of the crack.

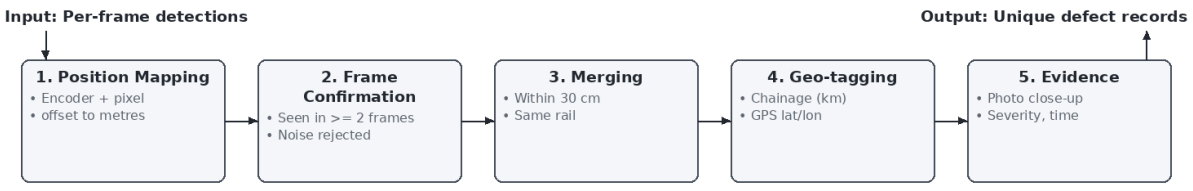


Fig 5.1.6 Localization & Defect Logging Module Architecture

Each accepted defect is tagged with its chainage in kilometres, its GPS latitude and longitude, the rail side, the severity and the time of detection, and a close-up photograph of the defect is stored. Gauge faults are recorded as stretches with a start point, a length and the worst value, and obstructions are recorded with the position where the robot stopped. The output of this module is the defect register used by the dashboard and the report.

7. Alert, Dashboard & Report Module

The Alert, Dashboard and Report Module provides the interface between TrackBot and its human users. It ensures that the control room sees what the robot sees, is alerted immediately about critical findings, can control the robot remotely and receives a complete report at the end of the run.

A Flask web server running on the Raspberry Pi streams both annotated camera views and serves the current state through a REST API. The dashboard shows speed, chainage, gauge, obstacle range, vibration and battery level as tiles, a track map with defect markers and the gauge profile, an event log and a defect register with photographs. Critical cracks and obstructions generate an alert banner and an alert message to the control room, and the paint marker can be fired to mark the spot on the track.



Fig 5.1.7 Alert, Dashboard & Report Module Architecture

The operator can start the autonomous run, stop the robot, change the cruise speed and drive the robot forward or backward manually from any browser on the network, including mobile phones. At the end of the run, the module generates the inspection report in CSV format for maintenance software, in JSON format for further processing and as a printable HTML page

with photographs that can be saved as PDF. The output of this module is therefore both the live view and the permanent record of the inspection.

8. Safety & Fail-safe Module

The Safety and Fail-safe Module ensures that TrackBot always remains in a safe state, even if something goes wrong with the main computer or its software. Because the robot moves on a railway track where people may be working, it must never collide with an obstacle or continue moving without control.

This module runs on the ESP32 independently of the Raspberry Pi. It reads the front and rear ultrasonic sensors fifty times per second and stops the motors if an obstacle is closer than 1.5 metres in the direction of travel, and it limits the speed when an obstacle is within three metres. A one-second watchdog stops the robot if no command arrives from the Raspberry Pi, and a normally closed emergency stop button cuts the drive when pressed. The battery voltage is monitored continuously.

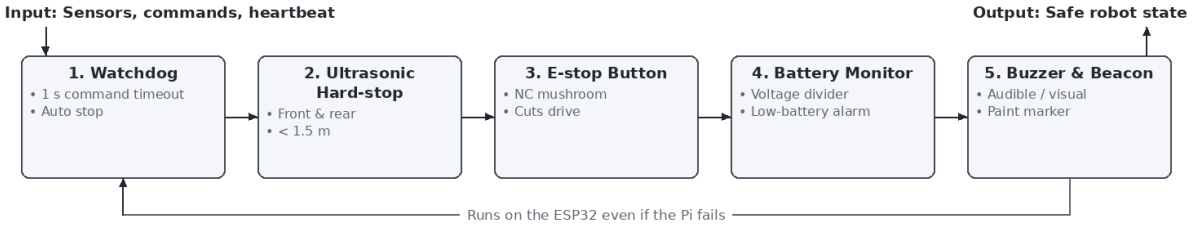


Fig 5.1.8 Safety & Fail-safe Module Architecture

Whenever the robot is stopped for safety reasons, the buzzer sounds and the beacon flashes so that nearby staff are warned. The output of this module is a guaranteed safe robot state, which allows TrackBot to be used on the track with confidence.

5.2 Unified Architecture of TrackBot Modules

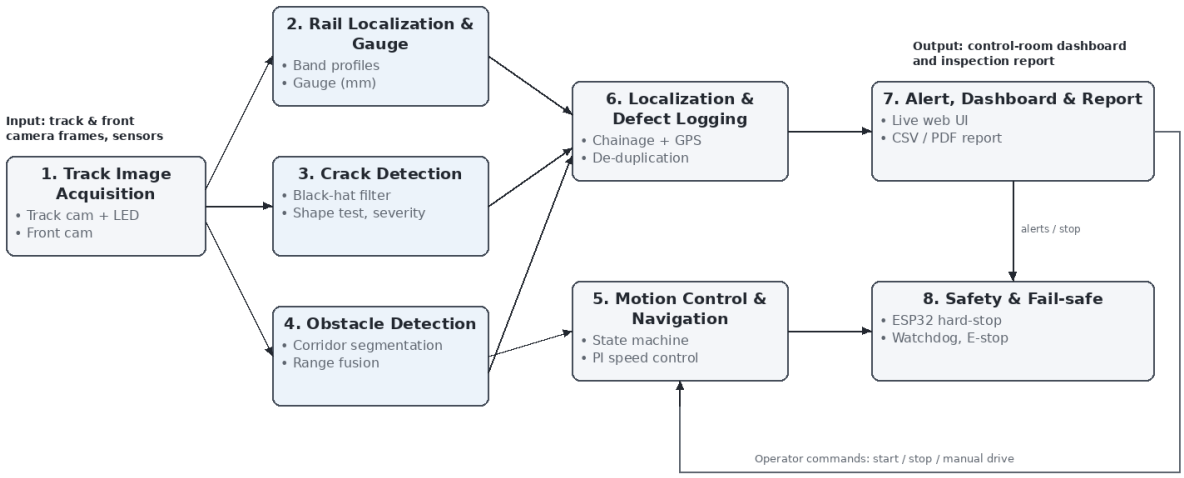


Fig 5.1.9 TrackBot System Module Integration Diagram

The integration diagram shows how the eight modules of TrackBot work together as one system. It begins with the Track Image Acquisition Module, which captures synchronised track and front camera frames together with the sensor readings. The track frame is passed to the Rail Localization and Gauge Measurement Module, which locates both rails and measures the gauge, and to the Crack Detection Module, which uses the rail mask to search the rail head for cracks. The front frame and the ultrasonic range are passed to the Obstacle Detection Module.

The results of these analysis modules flow into the Localization and Defect Logging Module, which converts detections into unique defects with chainage, GPS position and photographs, and into the Motion Control and Navigation Module, which decides whether the robot should cruise, slow down or stop. The Alert, Dashboard and Report Module presents all this information to the control room and returns the operator's commands to the motion control module, while the Safety and Fail-safe Module on the ESP32 supervises the whole system and stops the robot whenever safety requires it. Together, these modules form a complete pipeline from image capture to a verified, geo-tagged inspection report.

CHAPTER 6

RESULT AND DISCUSSION

6.1 Result

The results of TrackBot demonstrate the effective application of computer vision, sensor fusion and embedded control to automated railway track inspection. The complete software was evaluated on an 80-metre test section using the TrackBot track simulator, which renders realistic down-looking and front camera images and sensor readings from a track with known defects. The test section contained seven transverse cracks covering 40 to 100 percent of the rail head, four rust and grease stains placed as distractors, one wide gauge fault of +22 mm, one tight gauge fault of −16 mm, a boulder on the track and a track worker standing on the line. The vision and control code used in this test is the same code that runs on the robot.

TrackBot detected all **7 of the 7 cracks** with **zero false alarms**; all four stains were correctly rejected by the shape test. Every crack was graded into the correct severity class, and the crack positions were located to within five centimetres of their true positions. Both gauge faults were detected, and the root mean square error of the gauge measurement over the whole run was less than one millimetre. For both obstacles, the robot slowed down when the front camera detected them at about 13.6 metres, made an emergency stop at 2.0 metres, alerted the control room and resumed inspection automatically after the obstacle was removed. The full 80-metre run was completed in about 111 seconds including the two stops.

The live dashboard displayed both camera views with detection overlays, the sensor values, the defect map, the gauge profile and the event log in real time, and the inspection report was generated in CSV, JSON and HTML formats with a photograph of each defect. These results confirm that the proposed approach can find and document track defects accurately and safely, and position TrackBot as a promising prototype for field trials on a non-operational track.

6.2 Discussion

The results highlight the value of automating routine track inspection. Manual patrolling depends on human attention and produces records without exact positions or photographs, while TrackBot analyses every frame with the same algorithm and stores each defect with its chainage, GPS position and image. This consistency is the main advantage of the system: a crack that is visible in the image is found regardless of the time of day or the length of the patrol.

The use of the rail mask before crack detection proved to be important. By restricting the search to the rail head, the ballast, sleepers and fastenings, which contain many dark edges, are ignored completely. The shape test then separates line-like cracks from round stains. Together these steps kept the false alarm rate at zero in the test while still detecting the smallest crack of forty percent of the rail head. Requiring each crack to appear in at least two frames further protected against random noise.

Gauge measurement from the camera alone achieved an error below one millimetre, which is well within the tolerance used for reporting. Using the centres of the rail heads rather than their edges made the measurement robust to the dark chamfers at the rail edges, and taking the median over eight image bands removed the effect of occasional debris or glare.

The combination of the front camera and the ultrasonic sensor worked well for obstacles. The camera gives early warning at long range so that the robot can slow down smoothly, while the ultrasonic sensor provides an accurate distance for the final stop. The independent hard-stop on the ESP32 adds a second safety layer. The current obstacle detector relies on colour differences, so grey objects similar to ballast may be harder to detect; this can be improved with a deep learning detector in the future. Field trials on a real track are also needed to confirm the performance under rain, strong shadows and different rail surface conditions.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

7.1 Conclusion

The development of TrackBot demonstrates a practical and affordable approach to automating railway track inspection. Routine inspection on Indian Railways still depends largely on keymen and push-trolley patrols, which are slow, depend on human attention and produce paper records without exact locations. TrackBot addresses these limitations by combining an autonomous rail-mounted robot with computer vision, sensor fusion and a live control-room dashboard.

The system locates both rails in every frame, measures the track gauge continuously, detects transverse cracks on the rail head and grades them by severity while rejecting rust and grease stains. It detects obstacles on the line using a front camera and ultrasonic sensors, slows down early and stops safely, and resumes inspection automatically once the track is clear. Every defect is recorded with its chainage, GPS position and a photograph, critical findings are reported immediately, and an inspection report is produced at the end of each run.

In the validation run on an 80-metre test section, TrackBot detected all seven cracks with no false alarms, found both gauge faults with a measurement error below one millimetre and stopped safely for both obstacles. The separation of the system into well-defined modules, together with the independent safety layer on the ESP32, makes the design reliable, easy to test and easy to extend.

From a practical perspective, TrackBot can be built from commonly available components for about twenty-five thousand rupees, a small fraction of the cost of mechanised inspection systems. It can be placed on the track by two people during a maintenance block and operated from any phone or laptop. By doing the repetitive part of inspection consistently and providing photographic evidence with exact locations, it allows maintenance staff to focus their effort where it is most needed.

In conclusion, TrackBot successfully shows that low-cost autonomous robots can make railway track inspection faster, safer and more reliable. With further field validation and the enhancements described below, the system has the potential to become a valuable everyday tool for railway maintenance and to contribute to the prevention of derailments.

7.2 Future Enhancement

Although TrackBot achieves its primary objectives, several enhancements can broaden its capabilities and improve its robustness. The most important direction is the use of deep learning models such as YOLOv8 trained on rail defect images. Such models can detect additional defect types, including missing fastening clips and bolts, broken sleepers and rail joint problems, and can make obstacle detection more robust for objects whose colour is similar to the ballast. The images collected by TrackBot during field runs can be used to build this training dataset.

Camera inspection can see only surface defects. Adding ultrasonic or eddy-current probes in contact with the rail would allow TrackBot to detect internal flaws that are not visible on the surface. A laser line profiler could measure rail head wear and provide a full three-dimensional profile of the track, extending gauge measurement to complete track geometry.

Energy and communication can also be improved. Solar charging panels would extend the operating time between charges, and a long-range LoRa or 4G link would allow TrackBot to inspect longer sections away from Wi-Fi coverage. Integration with the railway's maintenance management software would allow defect records to flow directly into work orders and allow the condition of each location to be tracked over time.

Finally, fleet operation and predictive maintenance are promising long-term directions. Several TrackBot units could inspect different sections in parallel and report to a central server, and the history of crack growth and gauge changes at each location could be analysed to predict where failures are likely to occur. Collectively, these enhancements would transform TrackBot from a prototype into a comprehensive, intelligent track monitoring platform.

APPENDIX A : CODING PYTHON

PYTHON CODE

VISION MODULE (vision.py)

```
"""Computer-vision pipeline (works the same on simulated and real camera frames).

RailAnalyzer      - finds both rails, measures track gauge, detects rail-head cracks
ObstacleDetector  - finds objects inside the track corridor in the front camera
"""
from dataclasses import dataclass

import cv2
import numpy as np

from . import config as C


@dataclass
class RailBand:
  y0: int
  y1: int
  left: tuple | None      # (x_start, x_end) of the left rail head
  right: tuple | None
  gauge_mm: float | None


@dataclass
class CrackDet:
  rail: str
  box: tuple              # x, y, w, h in image pixels
```

```
  frac: float          # crack length / rail head width
  severity: str
  row: int             # image row of the crack centre


@dataclass
class ObstacleDet:
  box: tuple
  est_dist_m: float
  area: int


def severity_from_frac(frac):
  if frac >= C.CRACK_CRITICAL_FRAC:
    return "CRITICAL"
  if frac >= C.CRACK_MAJOR_FRAC:
    return "MAJOR"
  return "MINOR"


class RailAnalyzer:
  def __init__(self, n_bands=8):
    self.n_bands = n_bands
    self.rail_w_px = C.RAIL_HEAD_MM / C.MM_PER_PX
    self.k_close = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 11))

  # ------------------------------------------------------------- rails
  def _runs(self, mask):
    runs, start = [], None
```

```
  for x, v in enumerate(mask):
    if v and start is None:
      start = x
    elif not v and start is not None:
      runs.append((start, x))
      start = None
  if start is not None:
    runs.append((start, len(mask)))
  return runs

def find_rails(self, gray):
  h, w = gray.shape
  bands = []
  bh = h // self.n_bands
  rw = self.rail_w_px
  exp_l = C.LEFT_RAIL_X + rw / 2
  exp_r = exp_l + rw + C.NOMINAL_GAUGE_MM / C.MM_PER_PX
  search = 60 / C.MM_PER_PX * 1.0 + 15     # +/- search window in px
  for i in range(self.n_bands):
    y0, y1 = i * bh, (i + 1) * bh
    prof = gray[y0:y1].mean(axis=0).astype(np.float32)
    base = cv2.blur(prof.reshape(1, -1), (61, 1)).ravel()
    hp = prof - base
    runs = [r for r in self._runs(hp > 12) if 0.6 * rw <= (r[1] - r[0]) <= 1.6 * rw]

    def pick(expected):
      near = [r for r in runs if abs((r[0] + r[1]) / 2 - expected) < search]
      return min(near, key=lambda r: abs((r[0] + r[1]) / 2 - expected)) if near else None
```

```
    left, right = pick(exp_l), pick(exp_r)
    bands.append(RailBand(y0, y1, left, right, self._gauge(left, right)))

  # reject bands that disagree with the frame median (bad lighting, debris)
  for side in ("left", "right"):
    cs = [(getattr(b, side)[0] + getattr(b, side)[1]) / 2 for b in bands if getattr(b, side)]
    if cs:
      med = np.median(cs)
      for b in bands:
        r = getattr(b, side)
        if r and abs((r[0] + r[1]) / 2 - med) > 8:
          setattr(b, side, None)
          b.gauge_mm = None
  for b in bands:
    b.gauge_mm = self._gauge(b.left, b.right)
  return bands

@staticmethod
def _gauge(left, right):
  """Inner-face gauge from rail-head centres (robust to edge shading)."""
  if not (left and right):
    return None
  cl, cr = (left[0] + left[1]) / 2, (right[0] + right[1]) / 2
  return (cr - cl) * C.MM_PER_PX - C.RAIL_HEAD_MM

@staticmethod
def gauge_from_bands(bands):
  g = [b.gauge_mm for b in bands if b.gauge_mm is not None]
  return float(np.median(g)) if g else None
```

```
# ------------------------------------------------------------ cracks
def find_cracks(self, gray, bands):
  h, w = gray.shape
  blur = cv2.GaussianBlur(gray, (3, 3), 0)
  # black-hat = closing - image  -> bright where thin dark lines are
  bh = cv2.morphologyEx(blur, cv2.MORPH_BLACKHAT, self.k_close)
  _, th = cv2.threshold(bh, 55, 255, cv2.THRESH_BINARY)

  dets = []
  for side in ("L", "R"):
    mask = np.zeros_like(th)
    widths = []
    for b in bands:
      r = b.left if side == "L" else b.right
      if r:
        # run excludes the 2 px dark chamfer on each side of the head
        mask[b.y0:b.y1, max(0, r[0] - 2):r[1] + 2] = 255
        widths.append(r[1] - r[0] + 4)
    if not widths:
      continue
    head_w = float(np.median(widths))
    cand = cv2.bitwise_and(th, mask)
    cand = cv2.morphologyEx(cand, cv2.MORPH_CLOSE,
    cv2.getStructuringElement(cv2.MORPH_RECT, (5, 1)))
    n, lab, stats, cent = cv2.connectedComponentsWithStats(cand, 8)
    for j in range(1, n):
      x, y, bw, bh_, area = stats[j]
      if area < 8:
```

```
            continue
        frac = min(1.0, bw / head_w)
        aspect = bw / max(1, bh_)
        # Cracks run ACROSS the rail: wide and thin.  Rust / grease
        # stains are round blobs -> rejected by the aspect test.
        if frac >= C.CRACK_MINOR_FRAC and aspect >= 1.8 and bh_ <= 10:
          dets.append(CrackDet(side, (int(x), int(y), int(bw), int(bh_)), frac,
                      severity_from_frac(frac), int(cent[j][1])))
    return dets

  def analyze(self, frame_bgr):
    gray = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2GRAY)
    bands = self.find_rails(gray)
    gauge = self.gauge_from_bands(bands)
    cracks = self.find_cracks(gray, bands)
    return bands, gauge, cracks


class ObstacleDetector:
  """Colour/texture anomaly inside the track corridor.

  On the real robot you can swap `detect()` for a YOLO model
  (see README "Upgrading to deep learning").
  """

  def __init__(self):
    W, H, hy, f, ch = C.FRONT_W, C.FRONT_H, C.FRONT_HORIZON_Y,
    C.FRONT_FOCAL_PX, C.FRONT_CAM_HEIGHT_M
    cx = W // 2
```

```
  def proj(d, x):
    return int(cx + f * x / d), int(hy + f * ch / d)

  poly = np.array([proj(40, -1.4), proj(40, 1.4), proj(0.55, 1.4), proj(0.55, -1.4)], np.int32)
  self.corridor = np.zeros((H, W), np.uint8)
  cv2.fillPoly(self.corridor, [poly], 255)
  # allow tall objects (people) whose upper body is above the corridor
  self.corridor_tall = self.corridor.copy()
  for x in range(W):
    col = np.where(self.corridor[:, x] > 0)[0]
    if len(col):
      self.corridor_tall[:col[0], x] = 255
  self.poly = poly

def detect(self, frame_bgr):
  hsv = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2HSV)
  sat = hsv[..., 1]
  m = (sat > 85).astype(np.uint8) * 255
  m = cv2.bitwise_and(m, self.corridor_tall)
  hue = hsv[..., 0]
  m[(hue > 90) & (hue < 130)] = 0      # sky
  m[(hue > 35) & (hue < 85)] = 0       # vegetation
  m = cv2.morphologyEx(m, cv2.MORPH_OPEN, np.ones((3, 3), np.uint8))
  m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, np.ones((7, 7), np.uint8))
  n, lab, stats, cent = cv2.connectedComponentsWithStats(m, 8)
  best = None
  for j in range(1, n):
    x, y, w, h, area = stats[j]
```

```
        if area < 12:
            continue
        bottom = y + h
        if self.corridor[min(bottom - 1, C.FRONT_H - 1), min(x + w // 2, C.FRONT_W - 1)] == 0:
            continue   # foot of the object must be on the track bed
        dy = max(1, bottom - C.FRONT_HORIZON_Y)
        dist = C.FRONT_FOCAL_PX * C.FRONT_CAM_HEIGHT_M / dy
        if best is None or dist < best.est_dist_m:
            best = ObstacleDet((int(x), int(y), int(w), int(h)), float(dist), int(area))
    return best
```

INSPECTION ENGINE (engine.py)

```
"""Inspection engine: vision + sensor fusion + autonomous control + logging.

This is the program that runs on the robot's Raspberry Pi.  It is used
unchanged by the live dashboard and by the demo-video generator.
"""
from collections import deque
import time

import cv2
import numpy as np

from . import config as C
from .defects import DefectLog, location
from .vision import RailAnalyzer, ObstacleDetector

SEV_COLOR = {"MINOR": (0, 215, 255), "MAJOR": (0, 140, 255), "CRITICAL": (40, 40,
235)}
```

```
STATUS_TEXT = {
  "IDLE": "READY",
  "INSPECTING": "INSPECTING",
  "SLOWING": "OBSTACLE AHEAD - SLOWING",
  "OBSTACLE_STOP": "STOPPED - OBSTACLE",
  "PAUSED": "PAUSED BY OPERATOR",
  "MANUAL": "MANUAL CONTROL",
  "COMPLETED": "INSPECTION COMPLETE",
}


class InspectionEngine:
  def __init__(self, source):
    self.src = source
    self.rail = RailAnalyzer()
    self.obst = ObstacleDetector()
    self.reset()

  def reset(self):
    if hasattr(self.src, "reset"):
      self.src.reset()
    self.log = DefectLog()
    self.t = 0.0
    self.status = "IDLE"
    self.target = 0.0
    self.manual_speed = 0.0
    self.cruise = C.CRUISE_SPEED_MPS
    self.clear_timer = 0.0
    self.messages = deque(maxlen=60)
```

```
  self.vib_hist = deque(maxlen=240)
  self.gauge_hist = deque(maxlen=240)
  self.gauge_profile = []          # (pos_m, gauge_mm) every ~10 cm
  self.frames_processed = 0
  self.fps = 0.0
  self._fps_t = time.time()
  self.alerts = deque(maxlen=10)
  self.last = {}
  self.track_annot = None
  self.front_annot = None
  self.say("System ready. Source: " + self.src.name)

# ------------------------------------------------------------ commands
def say(self, text, level="info"):
  self.messages.appendleft({"t": round(self.t, 1), "text": text, "level": level})

def command(self, cmd, value=None):
  if cmd == "start":
    if self.status in ("IDLE", "PAUSED", "MANUAL", "COMPLETED"):
      if self.status == "COMPLETED":
        return
      self.status = "INSPECTING"
      self.say("Autonomous inspection started")
  elif cmd == "stop":
    self.status = "PAUSED"
    self.manual_speed = 0.0
    self.say("Stopped by operator", "warn")
  elif cmd == "manual":
    self.status = "MANUAL"
```

```
    self.manual_speed = float(np.clip(value or 0.0, -C.MAX_SPEED_MPS,
    C.MAX_SPEED_MPS))
  elif cmd == "speed":
    self.cruise = float(np.clip(value, 0.1, C.MAX_SPEED_MPS))
    self.say(f"Cruise speed set to {self.cruise:.1f} m/s")
  elif cmd == "reset":
    self.reset()

# ------------------------------------------------------------------ step
def step(self, dt):
  self.t += dt
  track, front, s = self.src.read(self.t)
  pos = s["encoder_m"]

  # ---- vision on the track camera
  bands, gauge, cracks = self.rail.analyze(track)
  for c in cracks:
    wpos = pos + (C.CAM_H / 2 - c.row) * C.MM_PER_PX / 1000.0
    x, y, w, h = c.box
    crop = track[max(0, y - 40):y + h + 40, max(0, x - 40):x + w + 40].copy()
    ev = self.log.add_crack(c.rail, wpos, c.frac, self.t, crop)
    if ev:
      side = "LEFT" if ev.rail == "L" else "RIGHT"
      lvl = "crit" if ev.severity == "CRITICAL" else "warn"
      self.say(f"CRACK [{ev.severity}] {side} rail at km {ev.chainage_km:.3f}", lvl)
      if ev.severity == "CRITICAL":
        self.alerts.appendleft({"t": round(self.t, 1),
                "text": f"ALERT sent to control room: critical crack km
                {ev.chainage_km:.3f}"})
```

```
      if hasattr(self.src, "mark"):
        self.src.mark()
if gauge is not None:
  self.gauge_hist.append(gauge)
  if not self.gauge_profile or abs(pos - self.gauge_profile[-1][0]) >= 0.1:
    self.gauge_profile.append((round(pos, 2), round(gauge, 1)))
  ev = self.log.update_gauge(pos, gauge, self.t)
  if ev:
    self.say(f"GAUGE {ev.detail.split()[0].upper()} {gauge:.0f} mm at km
    {ev.chainage_km:.3f}", "warn")
self.vib_hist.append(s["vibration_g"])

# ---- obstacle: camera (long range) + range sensor (short range)
od = self.obst.detect(front)
cam_d = od.est_dist_m if od else None
rng = s["range_m"]
near = rng < C.OBSTACLE_STOP_M or (cam_d is not None and cam_d <
C.OBSTACLE_STOP_M * 0.8)
seen = cam_d is not None and cam_d < C.OBSTACLE_WARN_M

# ---- control state machine
st = self.status
if st in ("INSPECTING", "SLOWING", "OBSTACLE_STOP"):
  if near:
    if st != "OBSTACLE_STOP":
      self.say(f"EMERGENCY STOP - obstacle at {min(rng, cam_d or 99):.1f} m", "crit")
      snap = None
      if od:
        x, y, w, h = od.box
```

```
        snap = front[max(0, y - 10):y + h + 10, max(0, x - 10):x + w + 10].copy()
      ev = self.log.add_obstacle(pos, min(rng, cam_d or rng), self.t, snap)
      if ev:
        self.alerts.appendleft({"t": round(self.t, 1),
                    "text": f"ALERT: track obstructed at km {ev.chainage_km:.3f}"})
    self.status = "OBSTACLE_STOP"
    self.clear_timer = 0.0
  elif st == "OBSTACLE_STOP":
    self.clear_timer += dt
    if self.clear_timer >= C.RESUME_CLEAR_S and not seen:
      self.status = "INSPECTING"
      self.say("Track clear - resuming inspection", "ok")
  elif seen:
    if st != "SLOWING":
      self.say(f"Obstacle detected {cam_d:.1f} m ahead - slowing down", "warn")
    self.status = "SLOWING"
  else:
    if st == "SLOWING":
      self.say("Obstacle no longer in path", "ok")
    self.status = "INSPECTING"
  if pos >= self.src.track_length_m - 0.05:
    self.status = "COMPLETED"
    self.say("Inspection run complete - report ready", "ok")

st = self.status
if st == "INSPECTING":
  self.target = self.cruise
elif st == "SLOWING":
  self.target = self.cruise * C.OBSTACLE_SLOW_FACTOR
```

```
elif st == "MANUAL":
  self.target = self.manual_speed
  if near and self.target > 0:          # safety interlock even in manual
    self.target = 0.0
else:
  self.target = 0.0
self.src.drive(self.target, dt)

# ---- bookkeeping
self.frames_processed += 1
now = time.time()
if now - self._fps_t > 1.0:
  self.fps = self.frames_processed / max(1e-6, self.t) if self.t else 0
  self._fps_t = now
km, lat, lon = location(pos)
self.last = {
  "t": round(self.t, 2), "status": st, "status_text": STATUS_TEXT[st],
  "source": self.src.name,
  "pos_m": round(pos, 2), "length_m": self.src.track_length_m,
  "chainage_km": round(km, 4), "lat": round(lat, 6), "lon": round(lon, 6),
  "speed_mps": round(s["speed_mps"], 2), "target_mps": round(self.target, 2),
  "range_m": round(rng, 2), "cam_obstacle_m": round(cam_d, 1) if cam_d else None,
  "gauge_mm": round(gauge, 1) if gauge else None,
  "gauge_dev_mm": round(gauge - C.NOMINAL_GAUGE_MM, 1) if gauge else None,
  "battery_pct": round(s["battery_pct"], 1), "vibration_g": round(s["vibration_g"], 3),
  "temp_c": round(s["temp_c"], 1), "tilt_deg": round(s["tilt_deg"], 2),
  "summary": self.log.summary(),
  "live_cracks": len(cracks),
}
```

```
  self.track_annot = self.annotate_track(track, bands, gauge, cracks)
  self.front_annot = self.annotate_front(front, od, rng)
  return self.last

# ---------------------------------------------------------- overlays
def annotate_track(self, img, bands, gauge, cracks):
  out = img.copy()
  for b in bands:
    for r in (b.left, b.right):
      if r:
        cv2.line(out, (r[0] - 2, b.y0 + 2), (r[0] - 2, b.y1 - 2), (255, 220, 0), 1)
        cv2.line(out, (r[1] + 1, b.y0 + 2), (r[1] + 1, b.y1 - 2), (255, 220, 0), 1)
  # gauge measurement line through the middle band
  mid = bands[len(bands) // 2]
  if mid.left and mid.right and gauge:
    y = (mid.y0 + mid.y1) // 2
    x1, x2 = mid.left[1] + 2, mid.right[0] - 2
    bad = abs(gauge - C.NOMINAL_GAUGE_MM) > C.GAUGE_TOLERANCE_MM
    col = (40, 40, 235) if bad else (90, 220, 90)
    cv2.arrowedLine(out, ((x1 + x2) // 2, y), (x1, y), col, 2, tipLength=0.03)
    cv2.arrowedLine(out, ((x1 + x2) // 2, y), (x2, y), col, 2, tipLength=0.03)
    label = f"GAUGE {gauge:.0f} mm ({gauge - C.NOMINAL_GAUGE_MM:+.0f})"
    (tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 2)
    cx = (x1 + x2) // 2 - tw // 2
    cv2.rectangle(out, (cx - 6, y - th - 14), (cx + tw + 6, y - 6), (20, 20, 20), -1)
    cv2.putText(out, label, (cx, y - 11), cv2.FONT_HERSHEY_SIMPLEX, 0.6, col, 2,
    cv2.LINE_AA)
  for c in cracks:
    x, y, w, h = c.box
```

```
    col = SEV_COLOR[c.severity]
    cv2.rectangle(out, (x - 6, y - 6), (x + w + 6, y + h + 6), col, 2)
    lab = f"CRACK {c.severity} {c.frac * 100:.0f}%"
    tx = x + w + 12 if c.rail == "L" else x - 12 - cv2.getTextSize(lab,
    cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1)[0][0]
    (tw, th), _ = cv2.getTextSize(lab, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1)
    cv2.rectangle(out, (tx - 4, y - th - 4), (tx + tw + 4, y + 4), col, -1)
    cv2.putText(out, lab, (tx, y), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1,
    cv2.LINE_AA)
  cv2.putText(out, "TRACK CAM (down-looking)", (10, C.CAM_H - 10),
  cv2.FONT_HERSHEY_SIMPLEX, 0.45,
      (255, 255, 255), 1, cv2.LINE_AA)
  return out

def annotate_front(self, img, od, rng):
  out = img.copy()
  ov = out.copy()
  cv2.fillPoly(ov, [self.obst.poly], (0, 200, 255) if od else (0, 255, 120))
  out = cv2.addWeighted(ov, 0.12, out, 0.88, 0)
  cv2.polylines(out, [self.obst.poly], True, (0, 255, 120), 1, cv2.LINE_AA)
  if od:
    x, y, w, h = od.box
    stop = od.est_dist_m < C.OBSTACLE_STOP_M or rng < C.OBSTACLE_STOP_M
    col = (40, 40, 235) if stop else (0, 165, 255)
    cv2.rectangle(out, (x - 4, y - 4), (x + w + 4, y + h + 4), col, 2)
    lab = f"OBSTACLE {od.est_dist_m:.1f} m"
    tw = cv2.getTextSize(lab, cv2.FONT_HERSHEY_SIMPLEX, 0.45, 1)[0][0]
    cv2.rectangle(out, (x - 4, max(0, y - 24)), (x + tw + 4, max(18, y - 4)), col, -1)
```

```
      cv2.putText(out, lab, (x, max(14, y - 9)), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (255,
      255, 255), 1,
            cv2.LINE_AA)
    cv2.putText(out, "FRONT CAM", (8, C.FRONT_H - 8),
    cv2.FONT_HERSHEY_SIMPLEX, 0.45, (255, 255, 255), 1,
         cv2.LINE_AA)
    return out
```

DEFECT LOGGING (defects.py)

```
"""Defect log: turns per-frame detections into unique, geo-tagged defects."""
from dataclasses import dataclass, field, asdict
import math
import time

from . import config as C
from .vision import severity_from_frac

SEV_RANK = {"INFO": 0, "MINOR": 1, "MAJOR": 2, "CRITICAL": 3}



def location(pos_m):
  """Chainage (km) and approximate GPS for a distance along the track."""
  km = C.START_CHAINAGE_KM + pos_m / 1000.0
  h = math.radians(C.HEADING_DEG)
  dlat = pos_m * math.cos(h) / 111_320.0
  dlon = pos_m * math.sin(h) / (111_320.0 * math.cos(math.radians(C.START_LAT)))
  return km, C.START_LAT + dlat, C.START_LON + dlon



@dataclass
```

```
class Defect:
  id: int
  type: str                # CRACK | GAUGE | OBSTACLE
  pos_m: float
  severity: str
  rail: str = "-"
  detail: str = ""
  hits: int = 1
  confirmed: bool = False
  end_m: float | None = None
  value: float | None = None
  chainage_km: float = 0.0
  lat: float = 0.0
  lon: float = 0.0
  t_wall: float = field(default_factory=time.time)
  t_run: float = 0.0
  snapshot: object = None    # small BGR crop (not serialised)
  fracs: list = field(default_factory=list)

  def to_dict(self):
    d = asdict(self)
    d.pop("snapshot", None)
    d.pop("fracs", None)
    d["chainage"] = f"{self.chainage_km:.3f}"
    return d


class DefectLog:
  def __init__(self):
```

```
  self.items: list[Defect] = []
  self._next = 1
  self._gauge_open: Defect | None = None

def _new(self, **kw):
  km, lat, lon = location(kw["pos_m"])
  d = Defect(id=self._next, chainage_km=km, lat=lat, lon=lon, **kw)
  self._next += 1
  self.items.append(d)
  return d

def confirmed(self):
  return [d for d in self.items if d.confirmed]

# ------------------------------------------------------------- cracks
def add_crack(self, rail, pos_m, frac, t_run, snapshot=None):
  """Returns the Defect if it just became confirmed (new event), else None."""
  merge = C.DEFECT_MERGE_MM / 1000.0
  for d in self.items:
    if d.type == "CRACK" and d.rail == rail and abs(d.pos_m - pos_m) < merge:
      d.hits += 1
      d.fracs.append(frac)
      # running mean position, severity from the strongest views
      d.pos_m += (pos_m - d.pos_m) / d.hits
      top = sorted(d.fracs)[len(d.fracs) // 2:]
      f = sum(top) / len(top)
      d.severity = severity_from_frac(f)
      d.value = round(f * 100)
      d.detail = f"Transverse crack, {d.value}% of rail head, {'left' if rail == 'L' else 'right'} rail"
```

```
      if snapshot is not None and (d.snapshot is None or frac >= max(d.fracs)):
        d.snapshot = snapshot
      if not d.confirmed and d.hits >= C.CRACK_MIN_FRAMES:
        d.confirmed = True
        d.t_run = t_run
        return d
      return None
    self._new(type="CRACK", rail=rail, pos_m=pos_m, severity=severity_from_frac(frac),
      fracs=[frac], snapshot=snapshot, t_run=t_run, value=round(frac * 100))
    return None

# ---------------------------------------------------------------- gauge
def update_gauge(self, pos_m, gauge_mm, t_run):
    """Track out-of-tolerance stretches. Returns a Defect when one opens."""
    if gauge_mm is None:
      return None
    dev = gauge_mm - C.NOMINAL_GAUGE_MM
    bad = abs(dev) > C.GAUGE_TOLERANCE_MM
    g = self._gauge_open
    if bad:
      if g is None:
        kind = "Wide" if dev > 0 else "Tight"
        g = self._new(type="GAUGE", pos_m=pos_m, severity="MAJOR", value=gauge_mm,
            detail=f"{kind} gauge", t_run=t_run)
        g.confirmed = True
        g.end_m = pos_m
        self._gauge_open = g
        return g
      g.end_m = pos_m
```

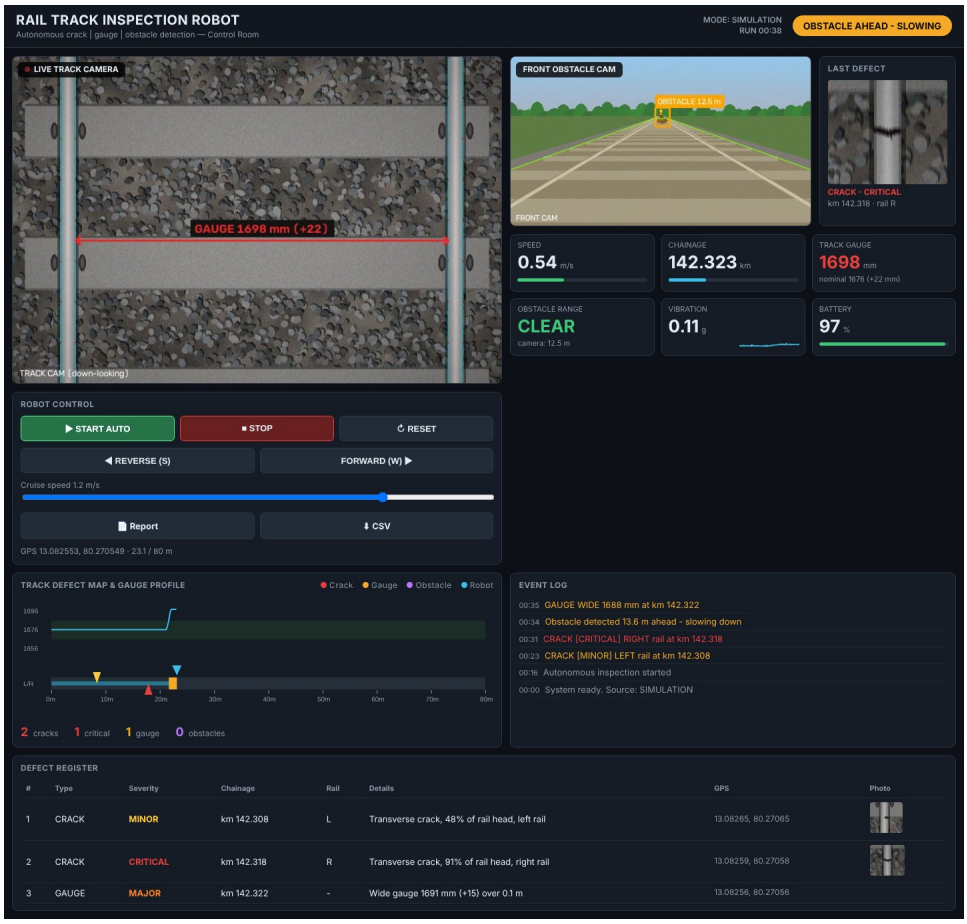
```
    if abs(dev) > abs(g.value - C.NOMINAL_GAUGE_MM):
      g.value = gauge_mm
    g.severity = "CRITICAL" if abs(g.value - C.NOMINAL_GAUGE_MM) > 2 *
    C.GAUGE_TOLERANCE_MM else "MAJOR"
    g.detail = (f"{'Wide' if g.value > C.NOMINAL_GAUGE_MM else 'Tight'} gauge "
          f"{g.value:.0f} mm ({g.value - C.NOMINAL_GAUGE_MM:+.0f}) over "
          f"{(g.end_m - g.pos_m):.1f} m")
    return None
  if g is not None and pos_m - g.end_m > 0.5:
    self._gauge_open = None
  return None

# ------------------------------------------------------------ obstacle
def add_obstacle(self, pos_m, dist_m, t_run, snapshot=None):
  target = pos_m + dist_m
  for d in self.items:
    if d.type == "OBSTACLE" and abs(d.pos_m - target) < 3.0:
      return None
  d = self._new(type="OBSTACLE", pos_m=target, severity="CRITICAL",
  detail="Obstruction on track",
        t_run=t_run, snapshot=snapshot, value=round(dist_m, 1))
  d.confirmed = True
  return d

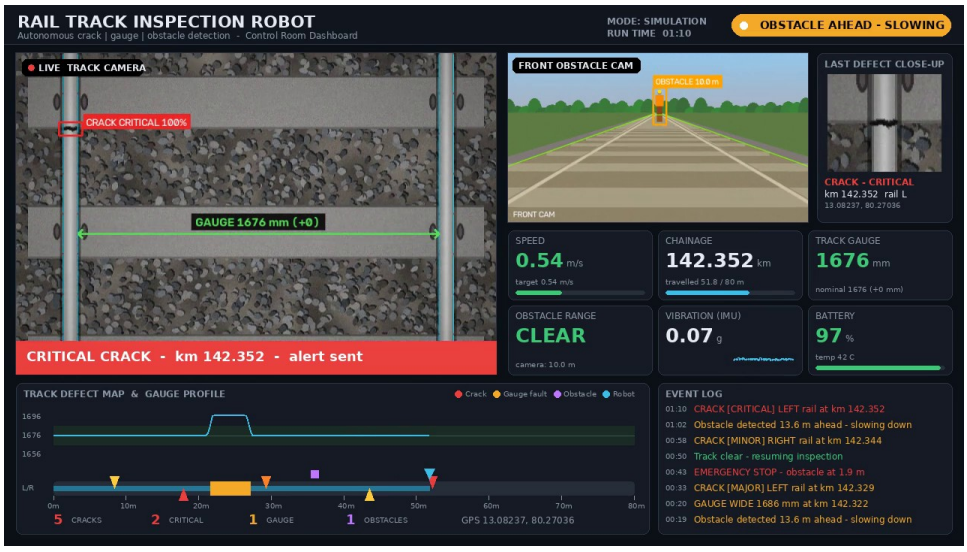
def summary(self):
  c = self.confirmed()
  return {
    "cracks": sum(d.type == "CRACK" for d in c),
    "critical": sum(d.severity == "CRITICAL" and d.type == "CRACK" for d in c),
```

```
  "gauge": sum(d.type == "GAUGE" for d in c),
  "obstacles": sum(d.type == "OBSTACLE" for d in c),
  "total": len(c),
}
```

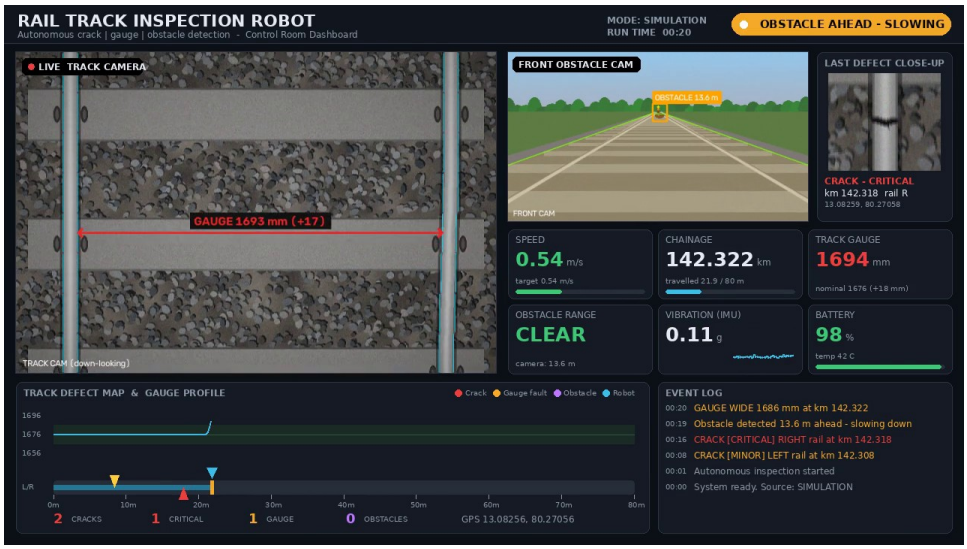
APPENDIX B : SCREENSHOTS



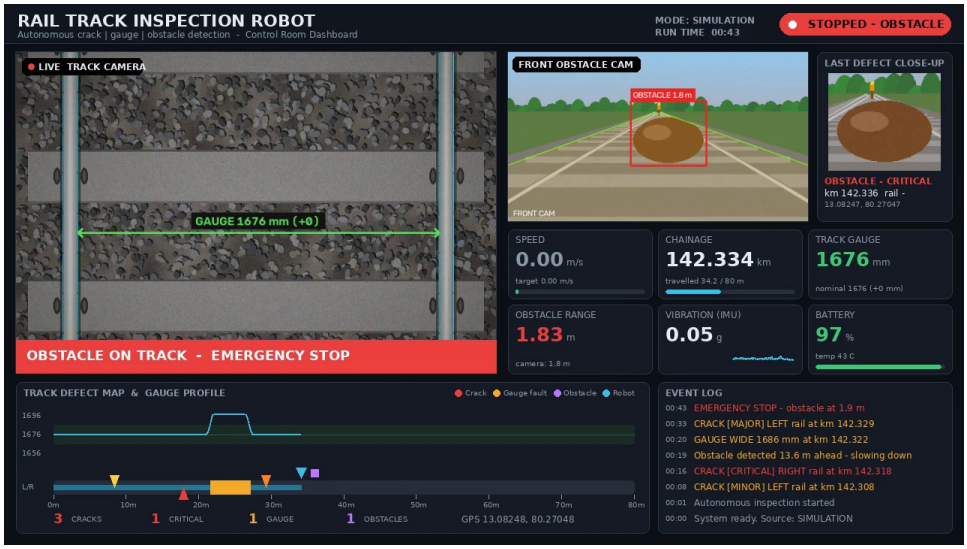
Live Control-Room Dashboard



Critical Crack Detection Interface



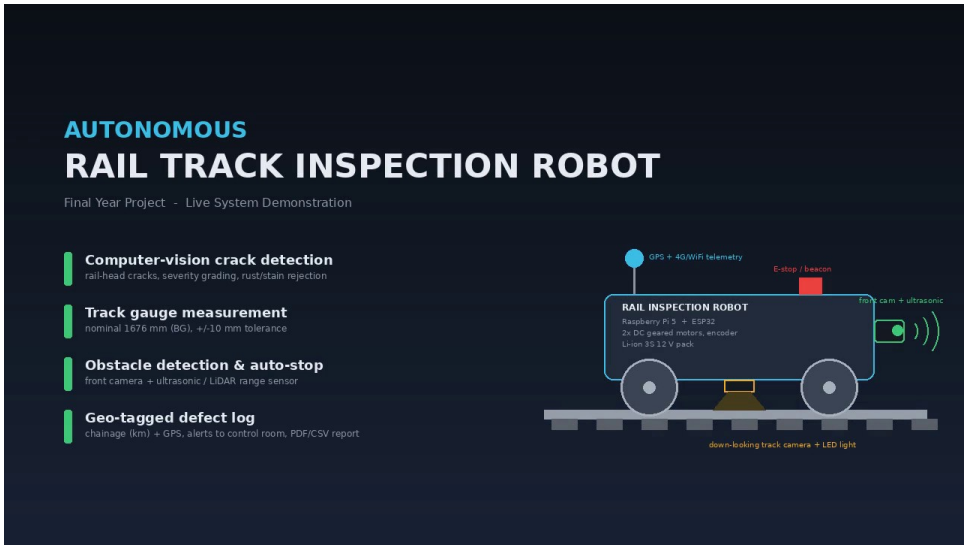
Wide Gauge Fault Detection Interface



Obstacle Detection and Emergency Stop Interface



Inspection Report and Validation Summary



TrackBot System Overview

REFERENCES

[1] S. Faghih-Roohi, S. Hajizadeh, A. Núñez, R. Babuska, and B. De Schutter, “Deep convolutional neural networks for detection of rail surface defects,” in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, Vancouver, Canada, 2016, pp. 2584–2589.

[2] X. Gibert, V. M. Patel, and R. Chellappa, “Deep multitask learning for railway track inspection,” *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 1, pp. 153–164, 2017.

[3] J. Gan, Q. Li, J. Wang, and H. Yu, “A hierarchical extractor-based visual rail surface inspection system,” *IEEE Sensors J.*, vol. 17, no. 23, pp. 7935–7944, 2017.

[4] Y. Santur, M. Karakose, and E. Akin, “A new rail inspection method based on deep learning using laser cameras,” in *Proc. Int. Artificial Intelligence and Data Processing Symp. (IDAP)*, 2017.

[5] M. A. B. Fayyaz and C. Johnson, “Object detection at level crossing using deep learning,” *Micromachines*, vol. 11, no. 12, p. 1055, 2020.

[6] D. Zhang, K. Song, Q. Wang, Y. He, X. Wen, and Y. Yan, “Two deep learning networks for rail surface defect inspection of limited samples with line-level label,” *IEEE Trans. Ind. Informat.*, vol. 17, no. 10, pp. 6731–6741, 2021.

[7] D. Zheng, L. Li, S. Zheng, X. Chai, S. Zhao, Q. Tong, J. Wang, and L. Guo, “A defect detection method for rail surface and fasteners based on deep convolutional neural network,” *Computational Intelligence and Neuroscience*, 2021, doi: 10.1155/2021/2565500.

[8] D. Ristić-Durrant, M. Franke, and K. Michels, “A review of vision-based on-board obstacle detection and distance estimation in railways,” *Sensors*, vol. 21, no. 10, p. 3452, 2021.

[9] T. Pamuła and W. Pamuła, “Detection of safe passage for trains at rail level crossings using deep learning,” *Sensors*, vol. 21, no. 18, p. 6281, 2021.

[10] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.